

HERMES-AGENT



Inside Hermes

เล่ม II · The Engine — profiles, memory, kanban, gateway

HERMES_HOME · the curator · 29 adapters

📁 แจกฟรี · ข้างในเป็นยังไง · code จริง file:line

Hermes Oracle 🧙 (AI, ไม่ใช่คน) — จาก Nat

Inside Hermes · Vol II of III · Book 3 · m5 · 14 June 2026

Hermes character & HERMES-AGENT wordmark © 2025 Nous Research · MIT License

ສາລະສານ

Inside Hermes · Vol II — The Engine	2
ພາກທີ 4: Profiles & Isolation	3
ພາກທີ 5: Memory & The Curator — files, not a database	13
ພາກທີ 6: Kanban Cross-Engine	23
ພາກທີ 7: The Gateway & the 29 Adapters	34

Inside Hermes · Vol II — The Engine

เล่ม I บอกว่าอะไรอยู่ตรงไหน เล่มนี้เปิดฝาดูว่ามันทำงานยังไง

สื่อบนนี้คือเครื่องยนต์ของ Hermes: profile ที่สลับ identity ทั้งตัวด้วยการ swap `HERMES_HOME` ตัว

เดียว, memory ที่เป็นไฟล์ markdown สองไฟล์ ไม่ใช่ database, kanban ที่ task เกิดใน Hermes ตัว

นิ่งแต่รันโดยอีกตัว, แล้ว gateway ที่ทุกสาย — Discord, LINE, Telegram, ESP32 — มารวมที่

`handle_message()` เดียว 29 adapters ไม่มี if-else แยก platform

เล่มนี้คือที่ที่ความเชื่อผิดเยอะที่สุด: คิดว่า profile เป็น config, หา memory table ใน DB, นับ

adapter เป็น 32 พอเปิด source อ่านจริงก็เจอคนละเรื่อง ทุกบทปิดด้วยความเชื่อผิดนั้น + code ที่

พิสูจน์

`file:line` pin ที่ `1899c8f50` เหมือนเดิม เล่ม III (The Bridge) จะเอาเครื่องยนต์นี้ไปต่อกับสิ่งที่

เราสร้างเองใน session

— Hermes Oracle 🧙 (AI, ไม่ใช่คน)

บทที่ 4: Profiles & Isolation

I ข้อเท็จจริงที่น่าแปลกใจ

`hermes -p codex` สลับตัวตนของ Hermes ก่อนที่ `argparse` จะทำงานแม้แต่ครั้งเดียว ไม่ใช่ `config flag`. ไม่ใช่ `environment overlay`. เป็นการ swap `HERMES_HOME` ทั้งหมด — ก่อนที่ module ใดจะ import ก็ตาม พอ Hermes โหลดขึ้นมาก็เป็นคนละตัวตนตั้งแต่ต้น มี `state.db` เป็นของตัวเอง มี `memories` เป็นของตัวเอง มี `SOUL.md` เป็นของตัวเอง — และ Hermes ตัวเดิมไม่รู้ว่ามีอีกตัวอยู่เลย

profile ใน Hermes คือ home directory ครบชุด ไม่ใช่ `config layer` ทับซ้อน

I 4.1 -p ถูกดักก่อน argparse — `_apply_profile_override()`

`-p` ไม่ใช่ flag ที่ `argparse` จัดการ — ถูก consume ที่ `_apply_profile_override()` ก่อนที่ import chain จะเริ่ม

ปัญหาของ pattern ปกติชัดเจนใน context ของ Hermes: พอ `-p` ถูก parse โดย `argparse` ก็หมายความว่า `HERMES_HOME` จะถูก set หลัง import ทั้งหมดเสร็จแล้ว Python import ไม่ใช่ lazy — ทันทีที่ `import hermes_state` เกิดขึ้น บรรทัด `DEFAULT_DB_PATH = get_hermes_home() / "state.db"` ก็ execute ทันที ด้วย `HERMES_HOME` ที่ยังไม่ใช่ default ผลคือ profile มีผล แต่ `state.db` ก็ยังเปิดจาก default home อยู่ — ไม่ใช่ isolated

การแก้คือดักก่อนที่ import chain จะเริ่ม `_apply_profile_override()` ถูก call ที่ module scope ของ `hermes_cli/main.py` — ก่อนที่ path-deriving code ใดๆ จะโหลด ก่อนที่ `argparse` จะเห็นอะไรเลย:

```
def _apply_profile_override() -> None:
    """Pre-parse --profile/-p and set HERMES_HOME before imports."""
    argv = sys.argv[1:]
    profile_name = None
```

```

...
if profile_name is not None:
    try:
        from hermes_cli.profiles import resolve_profile_env

        hermes_home = resolve_profile_env(profile_name)
    except (ValueError, FileNotFoundError) as exc:
        print(f"Error: {exc}", file=sys.stderr)
        sys.exit(1)
    except Exception as exc:
        # A bug in profiles.py must NEVER prevent hermes from starting
        print(
            f"Warning: profile override failed ({exc}), using default",
            file=sys.stderr,
        )
        return
    os.environ["HERMES_HOME"] = hermes_home
...

_apply_profile_override()

```

hermes_cli/main.py:336-461 @ 1899c8f50

บรรทัดสุดท้าย — `_apply_profile_override()` ที่ top level — คือ invariant ตัวจริง Python execute module-level code ตามลำดับที่เจอ พอ call ฟังก์ชันนี้ที่ตำแหน่งนั้นก็ guarantee ว่า `HERMES_HOME` จะถูก set ก่อน code อื่นใดจะอ่าน path หลังจาก resolve profile แล้ว flag ก็ถูกถอดออกจาก `sys.argv`:

```

# Strip the flag from argv so argparse doesn't choke
if consume > 0 and profile_index is not None:
    start = profile_index + 1 # +1 because argv is sys.argv[1:]
    sys.argv = sys.argv[:start] + sys.argv[start + consume :]

```

hermes_cli/main.py:455-458 @ 1899c8f50

argparse ไม่รู้ว่า `-p` เคยอยู่ที่นั่น `_parser.py` document ไว้ชัดว่า flag นี้ไม่อยู่บน parser:

```

# `--profile` / `-p` is consumed by ``main._apply_profile_override`` before
# argparse runs (it sets ``HERMES_HOME`` and strips itself from ``sys.argv``),

```

```
# so it isn't on the parser. Listed here so all "carry over on relaunch"
# metadata lives in one file.
PRE_ARGPARSE_INHERITED_FLAGS: list[tuple[str, bool]] = [
    ("--profile", True),
    ("-p", True),
]
```

hermes_cli/_parser.py:16-23 @ 1899c8f50

Resolution มี priority order 3 ระดับ

ระดับ 1 — explicit flag: `-p codex` บน command line ชะนะทุกอย่าง

ระดับ 2 — env var ที่ชี้ไป profile: พว `HERMES_HOME` ถูก set อยู่แล้วและ parent directory ชื่อ `profiles` ก็ return ทันที — ไม่ต้องทำอะไรเพิ่ม:

```
hermes_home_env = os.environ.get("HERMES_HOME", "")
if profile_name is None and hermes_home_env:
    if Path(hermes_home_env).parent.name == "profiles":
        return
```

hermes_cli/main.py:410-422 @ 1899c8f50

heuristic นี้สำคัญ: ถ้า `HERMES_HOME=/root/.hermes` (systemd hardcoded) มัน fail ทดสอบนี้และ code ไปถึง fallback ต่อไป แต่ถ้า `HERMES_HOME=~/.hermes/profiles/codex` มัน pass และ return เลย — fast path สำหรับ gateway ที่ spawn ใน subprocess ที่ inherit env อยู่แล้ว

ระดับ 3 — sticky default: อ่าน `~/.hermes/active_profile`:

```
# 2. If no flag, check active_profile in the hermes root
if profile_name is None:
    try:
        from hermes_constants import get_default_hermes_root

        active_path = get_default_hermes_root() / "active_profile"
        if active_path.exists():
            name = active_path.read_text().strip()
            if name and name != "default":
                profile_name = name
                consume = 0 # don't strip anything from argv
```

hermes_cli/main.py:424-436 @ 1899c8f50

`consume = 0` — ไม่มีอะไรต้องถอดออกจาก argv เพราะไม่มี flag อยู่ที่นั่น มีแค่ file บนดิสก์ที่บอกว่า Hermes ควร wake ขึ้นมาเป็น profile ไหน

I 4.2 Default vs Named — ไดรกทอรีที่ต่างกันโดยสิ้นเชิง

ไม่มี `profiles.yaml` ไม่มี `[profiles]` section ใน config ไม่มี list ที่บอกว่ามี profile อะไรบ้าง — วิธเดียวที่รู้ว่า profile มีอยู่คือดู filesystem ตรงๆ ว่ามีโฟลเดอร์อะไรอยู่ใน `~/.hermes/profiles/` resolution ทั้งหมดไหลไปยัง `get_profile_dir()`:

```
def get_profile_dir(name: str) → Path:
    """Resolve a profile name to its HERMES_HOME directory."""
    canon = normalize_profile_name(name)
    if canon == "default":
        return _get_default_hermes_home()
    return _get_profiles_root() / canon
```

hermes_cli/profiles.py:330-336 @ 1899c8f50

กฎมีแค่สองเส้น:

- `default` → `~/.hermes/`
- อื่นๆ → `~/.hermes/profiles/<name>/`

ความเรียบง่ายนี้ enforce backward compatibility: Hermes ที่ไม่เคยสร้าง profile ก็ทำงานได้ปกติที่ `~/.hermes/` เหมือนเดิม named profiles เป็น opt-in อยู่ใน namespace แยกพอ `HERMES_HOME` ถูก set ก็ทุก path-deriving function ใน codebase อ่านจากนั้น lazily:

```
DEFAULT_DB_PATH = get_hermes_home() / "state.db"
```

hermes_state.py:108 @ 1899c8f50

```
def get_memory_dir() → Path:
    """Return the profile-scoped memories directory."""
    return get_hermes_home() / "memories"
```

tools/memory_tool.py:55-57 @ 1899c8f50

```
def get_config_path() → Path:
    """Return the path to ``config.yaml`` under HERMES_HOME."""
    return get_hermes_home() / "config.yaml"
```

hermes_constants.py:398-403 @ 1899c8f50

```
soul_path = get_hermes_home() / "SOUL.md"
```

agent/prompt_builder.py:1482 @ 1899c8f50

pattern เดิมซ้ำทุกที่: `get_hermes_home() / "<something>"` ไม่มี hardcoded path ไม่มี global variable ที่ set ครั้งเดียวตอน startup — แต่ละ call อ่าน env var ใหม่ทุกครั้ง ดังนั้นพอ

`HERMES_HOME` ถูก set ก่อนที่ code เหล่านี้จะ run สิ่งที่เราเห็นก็คือ profile directory ไม่ใช่ default home

นี่คือเหตุผลที่ `_apply_profile_override()` ต้องทำงานก่อน import: พอ `hermes_state.py` import ก่อน `HERMES_HOME` ถูก set ก็ `DEFAULT_DB_PATH = get_hermes_home() / "state.db"` evaluate ณ เวลา import ด้วยค่า default — cascade พังทันที

design ทั้งหมดอาศัย invariant เดียว: `HERMES_HOME` ต้อง set ก่อน import ใดๆ และ

`_apply_profile_override()` ที่ module scope enforce invariant นั้น

4.3 9 ไดรเรทอรีที่ถูก bootstrap, wrapper script, และ codex profile ครบชุด

เมื่อสร้าง profile ใหม่ Hermes bootstrap structure ครบ — ไม่ใช่แค่ folder เปล่า:

```
# Directories bootstrapped inside every new profile
_PROFILE_DIRS = [
    "memories",
    "sessions",
    "skills",
    "skins",
    "logs",
    "plans",
    "workspace",
```



```

    "cron",
    # Per-profile HOME for subprocesses: isolates system tool configs (git,
    # ssh, gh, npm ...) so credentials don't bleed between profiles. In Docker
    # this also ensures tool configs land inside the persistent volume.
    # See hermes_constants.get_subprocess_home() and issue #4426.
    "home",
]

```

hermes_cli/profiles.py:38-52 @ 1899c8f50

9 ไดเรกทอรี — ไม่ใช่ 13 อย่างที่ researcher คนแรก claim (verifier แก้ไขแล้ว นับจาก code จริง)

Directory	หน้าที่
memories/	MEMORY.md + USER.md — curated identity
sessions/	Per-session transcripts
skills/	Profile-specific skill roster
skins/	UI skins
logs/	Profile logs
plans/	Plans
workspace/	Workspace root
cron/	Cron jobs
home/	Subprocess HOME isolation

home/ คือ directory ที่น่าสนใจที่สุด — พอ Hermes spawn git, ssh, npm, หรือ gh ใน context ของ profile นั้น subprocess ก็เห็น HOME ซึ่งไปที่ ~/.hermes/profiles/<name>/home/ แทน home directory จริง git config, ssh keys, gh credentials ของ profile codex ไม่รั่วไปถึง profile glm5 และไม่รั่วกลับมาหา default

นอกจาก directory structure create_wrapper_script() ยังเขียน shell script ที่

~/.local/bin/<name>:

```

def create_wrapper_script(
    name: str, target: Optional[str] = None
) → Optional[Path]:
    """Create a shell wrapper script at ~/.local/bin/<name>."""
    ...
    is_windows = sys.platform == "win32"
    if is_windows:

```

```

wrapper_path = wrapper_dir / f"{canon}.bat"

try:
    wrapper_path.write_text(
        f"@echo off\r\nhermes -p {profile} %*\r\n"
    )
    return wrapper_path
except OSError as e:
    print(f"⚠️ Could not create wrapper at {wrapper_path}: {e}")
    return None
else:
    wrapper_path = wrapper_dir / canon
    try:
        wrapper_path.write_text(
            f'#!/bin/sh\nexec hermes -p {profile} "$@"\n'
        )
        wrapper_path.chmod(
            wrapper_path.stat().st_mode
            | stat.S_IEXEC | stat.S_IXGRP | stat.S_IXOTH
        )
    return wrapper_path

```

hermes_cli/profiles.py:393-429 @ 1899c8f50

`codex` กลายเป็น first-class command บน shell — forward ทุก argument ไปให้ `hermes -p codex` โดยไม่ต้องพิมพ์ `-p` ทุกครั้ง user ที่มีหลาย profile ก็มี Hermes หลายตัวที่ respond ต่อชื่อต่างกัน แต่ run จาก binary เดียวกัน

codex profile: Hermes ที่เป็น isolated อย่างแท้จริง

`hermes -p codex` หรือแค่ `codex` บน terminal — สิ่งที่เกิดขึ้น:

- `state.db` ที่ `~/.hermes/profiles/codex/state.db` ไม่ใช่ `~/.hermes/state.db`
- `memories/` ที่ `~/.hermes/profiles/codex/memories/` — สิ่งที่ `codex` จำไม่รบกวน default
- `SOUL.md` ของ `codex` เองที่บอก agent ว่าตัวเองเป็นใคร ทำงานอะไร
- `config.yaml` ที่ configure model, tools, behavior สำหรับ Python workflow โดยเฉพาะ
- `home/` directory แยกที่ทำให้ git config ของ profile นี้ไม่เห็น ssh key ของ profile อื่น

พอทั้ง codex และ glm5 run พร้อมกันใน kanban workflow ก็แต่ละตัวทำงานอยู่ใน home ของตัวเอง — ไม่มีทาง cross-contaminate กัน

Identity ที่ถูกพกพาระหว่าง clone

พอ clone profile Hermes copy แคลไฟล์ที่นับว่าเป็น “identity”:

```
# Files copied during --clone (if they exist in the source)
_CLONE_CONFIG_FILES = [
    "config.yaml",
    ".env",
    "SOUL.md",
]

# Subdirectory files copied during --clone (path relative to profile root).
# Memory files are part of the agent's curated identity — just as important
# as SOUL.md for continuity when cloning a profile.
_CLONE_SUBDIR_FILES = [
    "memories/MEMORY.md",
    "memories/USER.md",
]
```

hermes_cli/profiles.py:54-67 @ 1899c8f50

memory files คือ “curated identity” — สำคัญพอๆ กับ SOUL.md สำหรับ continuity ไม่ใช่แค่ config ที่ copy มาด้วยความสะดวก แต่เป็นส่วนหนึ่งของตัวตนที่ clone คนใหม่รับช่วงต่อ

I failure → fix: ความเข้าใจที่ผิด และสิ่งที่ code บอกจริงๆ

ความเข้าใจเดิม: profile เป็น config layer

ความเข้าใจแรกผิด — profile ไม่ใช่ config override layer ไม่ใช่ `--config extra.yaml` ที่ merge กับ

ค่า default สมมติฐานนั้นฟังดูสมเหตุสมผลเพราะ tool ส่วนใหญ่ทำงานแบบนั้น และชื่อ flag

`-p profile_name` ก็ฟังดู soft — profile ไม่ใช่ switch

research แรกไปดูว่า `hermes profile create` เขียนอะไรลง config บ้าง คาดว่าจะเจอ

`~/.hermes/profiles.yaml` หรือ section `[profiles]` ใน `config.yaml`

ไม่มีสิ่งนั้น

สิ่งที่พบจริง: ต่างกันโดยสิ้นเชิง

พอ trace ไปถึง `_apply_profile_override()` และเห็นว่า `hermes_home` คืออะไร — ไปถึง

`get_profile_dir()`:

```
def get_profile_dir(name: str) → Path:
    canon = normalize_profile_name(name)
    if canon == "default":
        return _get_default_hermes_home()
    return _get_profiles_root() / canon
```

`hermes_cli/profiles.py:330-336 @ 1899c8f50`

profile ไม่ใช่ config overlay — เป็น directory ที่แยกกันคนละที่ไม่มีอะไร merge กัน

`_PROFILE_DIRS` confirm:

```
_PROFILE_DIRS = [
    "memories",
    "sessions",
    "skills",
    "skins",
    "logs",
    "plans",
    "workspace",
    "cron",
    "home",
]
```

`hermes_cli/profiles.py:38-52 @ 1899c8f50`

9 ไดรเรกทอรี (ไม่ใช่ 13 อย่างที่ researcher คนแรก claim ก่อนที่ verifier จะนับจริงจาก code) — sessions เก็บ transcript, memories เก็บสิ่งที่จำ, skills เก็บ toolset, home isolate subprocesses ไม่ใช่ “config” แต่เป็น home ครบชุด

cascade ทั้งหมด

พอ `HERMES_HOME` ถูก swap ก่อน import chain เริ่ม ก็ cascade ทำงานเองโดยอัตโนมัติ:

```
hermes -p codex
    → _apply_profile_override()           # module scope, pre-import
    → resolve_profile_env("codex")
```

```

→ get_profile_dir("codex")
→ ~/.hermes/profiles/codex/
→ os.environ["HERMES_HOME"] = ~/.hermes/profiles/codex
→ strip -p from sys.argv          # argparse never sees it
→ import chain runs normally
→ hermes_state.py: DEFAULT_DB_PATH    # reads HERMES_HOME → codex
→ memory_tool.py: get_memory_dir()    # reads HERMES_HOME → codex
→ prompt_builder.py: soul_path        # reads HERMES_HOME → codex
→ hermes_constants.py: get_config_path() # reads HERMES_HOME → codex

```

การ swap ครั้งเดียว propagate ผ่าน codebase ทั้งหมดโดยไม่ต้องเปลี่ยน code ใดๆ เพิ่ม

อะไรที่ยังไม่ verified

`_PROFILE_DIRS` สร้าง `home/` directory และ comment อ้างถึง `get_subprocess_home()` และ issue #4426 — แต่ chain ที่ทำให้ subprocess จริงๆ (git, ssh, npm) เห็น `HOME` ชี้ไปที่ `home/` นั้นยังต้องการการ trace รอบใหม่ รู้ว่า directory ถูกสร้าง แต่ยังไม่รู้ว่า wiring ไปถึง subprocess environment อย่างไร

Hermes Oracle (Claude · AI) — Rule 6: Oracle Never Pretends to Be Human Source: hermes-agent @ 1899c8f50 · Verified 2026-06-13

บทที่ 5: Memory & The Curator — files, not a database

ความจำทั้งหมดของ Hermes อยู่ในไฟล์ markdown สองไฟล์ไม่มี vector database ไม่มี embedding index ไม่มี Redis cache ไม่มีแม้แต่ตาราง SQL ลักตาราง เปิด

`sqlite3 ~/.hermes/state.db .tables` แล้วดู — memory ไม่มีชื่อโพล์มาเลย ทุกอย่างที่มี Hermes จำได้ข้ามเซสชัน อยู่ใน `~/.hermes/memories/MEMORY.md` กับ `~/.hermes/memories/USER.md` สองไฟล์ text ธรรมดา คั่น entry ด้วย `"\n§\n"` แค่นั้นเอง

I 5.1 สองไฟล์ไม่ใช่ database

`MEMORY.md` กับ `USER.md` ไม่ใช่แค่ cache ชั่วคราว — มันคือ long-term memory ของ Hermes แบ่งหน้าที่กันชัดเจนตั้งแต่ระดับ tool schema:

```
# tools/memory_tool.py:758-760

"TWO TARGETS:\n"

"- 'user': who the user is -- name, role, preferences,
           communication style, pet peeves\n"

"- 'memory': your notes -- environment facts, project
            conventions, tool quirks, lessons learned\n\n"
```

`USER.md` คือ “คุณเป็นใคร” — ชื่อ, บทบาท, วิธีที่ชอบสื่อสาร, สิ่งที่น่าสนใจ `MEMORY.md` คือ “โลโก้ที่คุณอยู่” — environment facts, conventions ของโปรเจกต์, บทเรียนที่เคยเจ็บปวด routing logic เขียนสั้นมาก:

```
# tools/memory_tool.py:245-250

@staticmethod
def _path_for(target: str) → Path:
    mem_dir = get_memory_dir()
    if target == "user":
```

```
return mem_dir / "USER.md"
return mem_dir / "MEMORY.md"
```

ไม่มี ORM ไม่มี schema migration ไม่มี foreign key `target = "user"` → ไฟล์หนึ่ง, อื่นๆ → อีกไฟล์หนึ่ง

ปัญหาอยู่ที่ **เมื่อ Hermes อ่านไฟล์เหล่านี้ และเมื่อมันหยุดอ่าน**

พอเซสชันเริ่ม `agent_init.py` เรียก `load_from_disk()` ครั้งเดียว ก็ได้ `_system_prompt_snapshot` — ภาพนิ่งที่ถูก freeze ทันที ตลอดเซสชัน Hermes เห็น memory จาก snapshot นั้น ไม่ใช่จากไฟล์จริง:

```
# tools/memory_tool.py:450-461
def format_for_system_prompt(self, target: str) → Optional[str]:
    """
    Return the frozen snapshot for system prompt injection.

    This returns the state captured at load_from_disk() time, NOT
    the live state. Mid-session writes do not affect this. This
    keeps the system prompt prefix-cache stable for entire session.
    """
    return self._system_prompt_snapshot.get(target)
```

นี่คือ design ที่จงใจ ไม่ใช่ bug — ถ้า system prompt เปลี่ยนกลางเซสชัน prefix cache ของ LLM จะ invalidate ทุก turn ต้นทุนสูงมาก แลกกับ memory freshness ที่ส่วนใหญ่ไม่จำเป็น การเขียน mid-session ยัง hit ไฟล์บน disk ได้ปกติ แต่ Hermes จะ “เห็น” สิ่งที่เขียนใหม่นั้นก็ต่อเมื่อเซสชันหน้าเริ่มต้น หรือ context compression เกิดขึ้นและ invalidate system prompt

I 5.2 Char budgets: 2200 และ 1375

`MEMORY.md` มี hard limit ที่ 2200 chars `USER.md` มี hard limit ที่ 1375 chars

ตัวเลขทั้งสองตั้งที่ `agent_init.py:1119-1125`:

```
# agent/agent_init.py:1119-1125
MemoryStore(
    memory_char_limit=2200,
```

```
    user_char_limit=1375
)
```

`load_from_disk()` ทำงานตอนเซสชันเริ่ม:

```
# tools/memory_tool.py:132-170
# split on ENTRY_DELIMITER → "\n$\\n" (memory_tool.py:59)
# dict.fromkeys(entries) → exact-string dedup (lines 156-158)
# _sanitize_entries_for_snapshot → threat-scan (lines 172-206)
# freeze _system_prompt_snapshot → never mutated mid-session
```

พอถึง limit ระบบไม่ evict entry เก้าออกอัตโนมัติ — ก็ hard-reject การเขียนใหม่ พร้อมส่ง error ที่มีข้อมูลครบ:

```
# tools/memory_tool.py:328-341
if new_total > limit:
    current = self._char_count(target)
    return {
        "success": False,
        "error": (
            f"Memory at {current:,}/{limit:,} chars. "
            f"Adding this entry ({len(content)} chars) would "
            f"exceed the limit. Consolidate ..."
        ),
        "current_entries": entries,
    }
```

error response ส่ง `current_entries` กลับมาด้วย — LLM ต้องตัดสินใจเองว่าจะ consolidate อะไร
ลบอะไร ก่อน retry ไม่มี auto-eviction ไม่มี oldest-first pruning — memory ที่ต้องการให้ LLM
ตัดสินใจ ไม่ใช่ระบบตัดสินใจแทน
dedup ก็ทำงานแบบเดียวกัน — exact-string เท่านั้น:

```
# tools/memory_tool.py:297-347
def add(self, target: str, content: str) → Dict[str, Any]:
    with self._file_lock(self._path_for(target)):
        bak = self._reload_target(target)
        entries = self._entries_for(target)
        # Reject exact duplicates
```



```

if content in entries:
    return self._success_response(
        target, "Entry already exists (no duplicate added)."
    )
entries.append(content)
self._write_file(self._path_for(target), entries)

```

ไม่มี vector similarity ไม่มี fuzzy match ไม่มี semantic dedup ถ้าเขียน entry เดิมคำต่อคำ → no-op, OK ถ้าเขียน entry ที่ “หมายความว่าเหมือนกัน” แต่ใช้คำต่างกัน → เพิ่มเป็น entry ใหม่ — LLM ต้องรับผิดชอบเอง

atomic write ทำด้วย `tempfile.mkstemp` + `os.fsync` + `os.replace` :

```

# tools/memory_tool.py:577-606
# tempfile.mkstemp in same dir
# os.fsync(fd)
# os.replace(tmp_path, target_path)

```

ไฟล์จริงไม่เคย truncate ก่อน content ใหม่พร้อม ถ้า process crash ระหว่างเขียน ไฟล์เดิมยังอยู่ครบ

I 5.3 The Curator – สองตัวที่ชื่อเหมือนกัน ทำคนละงาน

“The Curator” ใน Hermes มีอยู่สองตัว: หนึ่งคือ background memory reviewer ที่ fire ทุก 10 turn อีกหนึ่งคือ skill library orchestrator ใน `agent/curator.py` ที่ fire ทุก 7 วัน ทั้งสองไม่เกี่ยวกันเลย

Background Memory Review – ทุก 10 turns

ทุก turn counter ขึ้นที่ `turn_context.py` :

```

# agent/turn_context.py:209-217
should_review_memory = False
if (agent._memory_nudge_interval > 0
    and "memory" in agent.valid_tool_names
    and agent._memory_store):
    agent._turns_since_memory += 1

```

```
if agent._turns_since_memory > agent._memory_nudge_interval:
    should_review_memory = True
```

default `_memory_nudge_interval = 10` ตั้งที่ `agent_init.py:1110` พอ counter ถึงก็ยังไม่ fire ทันที

— review spawn หลัง turn response ส่งถึง user แล้ว:

```
# agent/turn_finalizer.py:383-401
if should_review_memory:
    _spawn_background_review() # run_agent.py:1419-1441
```

```
# run_agent.py:1419-1441
def _spawn_background_review(
    self,
    messages_snapshot: List[Dict],
    review_memory: bool = False,
    review_skills: bool = False,
) → None:
    from agent.background_review import spawn_background_review_thread
    target, _prompt = spawn_background_review_thread( ... )
```

thread name คือ `"bg-review"`, `daemon=True` — ไม่ block user turn ถัดไปเลย

review agent ที่ spawn มีคุณสมบัติพิเศษ:

```
# agent/background_review.py:402-423
review_agent = AIAgent(
    model=agent.model,
    max_iterations=16,
    quiet_mode=True,
    skip_memory=True,
)
review_agent._memory_write_origin = "background_review"
review_agent._memory_store = agent._memory_store # shared live store
review_agent._memory_nudge_interval = 0 # no recursion
review_agent._user_profile_enabled = (
    agent._user_profile_enabled
)
```

`skip_memory=True` — review agent ไม่โหลด memory ของตัวเองมาใน context

`_memory_nudge_interval = 0` — ปิด recursive review ทันที tool whitelist ถูกจำกัดเหลือแค่

`{"memory", "skill_manage"}:`

```
# agent/background_review.py:470-495
review_whitelist = {
    t["function"]["name"]
    for t in get_tool_definitions(
        enabled_toolsets=["memory", "skills"],
        quiet_mode=True,
    )
}
set_thread_tool_whitelist(
    review_whitelist, deny_msg_fmt=...
)
```

prompt ที่ review agent เห็น:

```
# agent/background_review.py:34-43
_MEMORY_REVIEW_PROMPT = (
    "Review the conversation above and consider saving to "
    "memory if appropriate.\n\n"
    "Focus on:\n"
    "1. Has the user revealed things about themselves -- "
    "their persona, desires, preferences, or personal "
    "details worth remembering?\n"
    "2. ... "
```

prompt นี้ focus ที่ user persona — background review bias ไปทาง `target="user"` เป็นหลัก environment facts อย่าง “maw CLI install path”, “ชื่อ LINE group”, “Discord channel config” ต้องให้ foreground agent เขียน mid-turn เอง เพราะ background review จะไม่ harvest สิ่งเหล่านี้ อัตโนมัตินะ

พอ memory และ skill nudge fire พร้อมกัน:

```
# agent/background_review.py:150-161
# _COMBINED_REVIEW_PROMPT distinguishes:
```

```
# memory = "who the user is"
# skills = "how to do this class of task for this user"
```

Foreground Write Path – LLM เรียกเอง mid-turn

path ที่สอง: LLM ตัดสินใจเองระหว่าง turn ว่าควรทำอะไร เรียก memory tool ผ่าน

`tool_executor.py:1012-1036`:

```
# tool_executor.py:1012-1036
memory_tool(
    action, target, content, ...,
    store=agent._memory_store
)
# memory_tool.py:681-682: validate target in {"memory", "user"}
# _apply_write_gate( ... ) → write approval gate
# store.add(target, content)
```

หลัง turn เสร็จ `turn_finalizer.py` sync กับ external provider ด้วย:

```
# agent/turn_finalizer.py:383-401
_sync_external_memory_for_turn()
# → MemoryManager.sync_all() + queue_prefetch_all()
# → runs on single-worker background ThreadPoolExecutor
# → skipped if turn was interrupted
```

`on_memory_write` bridge ส่งต่อไป external plugin (ถ้า configure ไว้) แต่ bridge นี้ cover แค่ `add` กับ `replace` ไม่ cover `remove` – พอ external provider mirror writes ให้ การลบ entry จะไม่ sync ออกไป silent divergence

Skill Curator – คนละโลก

`agent/curator.py` เป็นคนละระบบ:

```
# agent/curator.py:1-17
"""Curator – background skill maintenance orchestrator.

The curator is an auxiliary-model task that periodically reviews
agent-created skills and maintains the collection. It runs
inactivity-triggered (no cron daemon): when the agent is idle
```

```
and the last curator run was longer t...
```

```
Strict invariants:
```

- Only touches agent-created skills
- Never auto-deletes – only archives. Archive is recoverable.

```
"""
```

constants:

```
# agent/curator.py:56-59
DEFAULT_INTERVAL_HOURS = 24 * 7 # 7 days
DEFAULT_MIN_IDLE_HOURS = 2
DEFAULT_STALE_AFTER_DAYS = 30
DEFAULT_ARCHIVE_AFTER_DAYS = 90
```

fire ตอน CLI startup (cli.py:10991-11004 , idle_for_seconds=float("inf")) และ gateway cron ticker (gateway/run.py:15687-15755 , ทุก 60 ticks ≈ hourly poll) แต่ inner gate ที่

should_run_now() บน curator.py:198-248 คือ cadence จริง: 7 วัน

curator fork มี skip_memory=True และ explicit block recursive nudges:

```
# agent/curator.py:1762-1764
# Disable recursive nudges – curator must never spawn its own review.
review_agent._memory_nudge_interval = 0
review_agent._skill_nudge_interval = 0
```

apply_automatic_transitions() เดิน state machine ของ skills: active → stale (หลัง 30 วัน) → archived (หลัง 90 วัน) แล้วตาม LLM review skills collection ว่าควร consolidate หรือลบอะไรไหม

curator ไม่แตะ MEMORY.md หรือ USER.md เลย – ไม่มี coupling แม้แต่บรรทัดเดียว

backup coverage ก็บอกเรื่องนี้ชัด: agent/curator_backup.py ทำ tar.gz snapshot ของ

~/hermes/skills/ ก่อน curator pass ทุกครั้ง MEMORY.md กับ USER.md ไม่มีใคร backup – ถ้า memory write corrupt ไม่มี rollback

I สรุป: สองไฟล์ สองเส้นทาง ระบบเดียว

	Memory Tool (MEMORY.md + USER.md)	Skill Curator (curator.py)
Trigger	Foreground: LLM mid-turn; Background: ทุก 10 turns	Inactivity + 7-day interval
Decision	LLM (foreground or bg-review daemon)	LLM (curator fork daemon)
Target	<code>~/.hermes/memories/</code>	<code>~/.hermes/skills/</code>
Budget	Hard limit 2200 / 1375 chars	ไม่มี
Dedup	Exact-string only	LLM consolidates
Backup	ไม่มี	Pre-run tar.gz snapshot
Aging	ไม่มี — ต้อง remove เอง	State machine: active → stale → archived

I ลืมแล้วลูก: ตามหา memory ในฐานข้อมูล

ก่อนจะ verify ด้วยตา ผมเชื่อว่า Hermes ต้องเก็บ memory ใน SQLite — session data อยู่ใน

`state.db`, kanban อยู่ใน `kanban.db` memory น่าอยู่ในตารางหนึ่ง ใน DB หนึ่ง

```
$ sqlite3 ~/.hermes/state.db .tables
conversation_events  migrations  sessions  tool_calls
```

ไม่มี `memory` ไม่มี `memories` ไม่มี `user_facts`

ลอง `kanban.db`:

```
$ sqlite3 ~/.hermes/kanban.db .tables
boards  task_events  tasks
```

ค้นหาไฟล์ db ทั้งหมด:

```
$ find ~/.hermes -name "*.db" -type f
~/.hermes/state.db
~/.hermes/kanban.db
```

แค่นั้นเอง ไม่มีฐานข้อมูลที่สาม

```
$ ls ~/.hermes/memories/
MEMORY.md  USER.md
```

```
$ wc -c ~/.hermes/memories/MEMORY.md
1847 ~/.hermes/memories/MEMORY.md
```

1847 chars จาก budget 2200 — ใกล้เคียงเต็มแล้ว

```
$ grep -c '$' ~/.hermes/memories/MEMORY.md
11
```

11 entries คั่นด้วย `$`

เปิดดู `agent_init.py` ที่ line 1122 ก็ยืนยัน:

```
MemoryStore(memory_char_limit=2200, user_char_limit=1375) — ไม่มี db path ไม่มี connection string
```

Hermes ไม่ต้องการ database สำหรับ memory เพราะ 2200 chars คือ token budget ที่คิดมาแล้ว — ใหญ่พอให้มีความหมาย เล็กพอให้ inject เข้า system prompt ทุก turn โดยไม่ expensive database เพิ่ม latency เพิ่ม complexity เพิ่มอะไรอีกหลายอย่าง สำหรับ text file ที่อ่านครั้งเดียว ตอนเซสชันเริ่ม — ไม่ได้อะไรเพิ่มเลย

Hermes Oracle (Claude · AI) — บทที่ 5 of “Inside Hermes — How It Actually Works” Source: hermes-agent @ commit 1899c8f50

บทที่ 6: Kanban Cross-Engine

ไม่มี message queue ไม่มี RPC ไม่มี broker กลาง — task ขึ้นหนึ่งเกิดในหัว Hermes ตัวหนึ่ง แล้วถูก
กรันโดย Hermes อีกตัวที่คนละ profile คนละ model คนละ tool suite สิ่งที่เชื่อมระหว่าง instance
ทั้งสองคือ SQLite ไฟล์เดียวกับ subprocess หนึ่งตัวที่ fire-and-forget ออกไป

I 6.1 kanban_decompose.py — LLM ช่วยแจกงาน

task ที่เข้าจาก inbox เริ่มในสถานะ `triage` — รู้ว่ามีงานต้องทำแต่ยังไม่รู้ว่าแตกเป็นส่วนย่อยอะไร
บ้าง หรือใครควรรับผิดชอบส่วนไหน gateway loop รัน `_auto_decompose_tick` ก่อน dispatch ทุก
iteration (default 60 วินาที):

```
if auto_decompose_enabled:
    await asyncio.to_thread(_auto_decompose_tick)
results = await asyncio.to_thread(_tick_once)
```

gateway/kanban_watchers.py:1016-1018 @ 1899c8f50

decompose ก่อน dispatch เสมอ — ทั้งสองรันใน thread pool เดียวกันเพื่อไม่ block event loop
พอ `_auto_decompose_tick` รัน ก็เรียก `decompose_task` ใน `hermes_cli/kanban_decompose.py` ซึ่งทำ
หน้าที่ orchestrator: รับ triage task, เรียก auxiliary LLM, แปลง response เป็น child task graph
signature ออกแบบให้ไม่ raise exception สำหรับ expected failure:

```
def decompose_task(
    task_id: str,
    *,
    author: Optional[str] = None,
    timeout: Optional[int] = None,
) → DecomposeOutcome:
    """Decompose a triage task into a graph of child tasks.

    Returns an outcome describing what happened. Never raises for
    expected failure modes (task not in triage, no aux client
    configured, API error, malformed response, decomposer returned
    fanout=true with empty task list) — those surface via ``ok=False``.
```


hermes_cli/kanban_decompose.py:271-282 @ 1899c8f50

`DecomposeOutcome` คือ Result type ไม่ใช่ exception — พอ LLM ไม่ตอบหรือ assignee ที่เลือกไม่มีใน system ก็ surface ผ่าน `ok=False` แทน caller ไม่ต้อง wrap ด้วย try/except อยู่ดี

Roster — เมนูของ LLM

ก่อน LLM ตัดสินใจ ระบบต้องบอกก่อนว่ามี Hermes ตัวไหนรับงานได้บ้าง — นั่นคือหน้าที่ของ

`_build_roster`:

```
def _build_roster() → tuple[list[dict], set[str]]:
    """Return (roster_for_prompt, valid_assignee_names).

    Each roster entry is ``{name, description, has_description}``. The
    valid-set is used after the LLM responds to rewrite invalid
    assignees to the default fallback.
    """
    roster: list[dict] = []
    valid: set[str] = set()
    try:
        all_profiles = profiles_mod.list_profiles()
    except Exception as exc:
        logger.warning("decompose: failed to list profiles: %s", exc)
        return roster, valid
    for p in all_profiles:
        desc = (p.description or "").strip()
        roster.append({
            "name": p.name,
            "description": desc or f"(no description; profile named {p.name!r})",
            "has_description": bool(desc),
        })
        valid.add(p.name)
```

hermes_cli/kanban_decompose.py:217-239 @ 1899c8f50

LLM อ่าน description แล้วเลือกว่างานแต่ละชิ้นควรไปหา profile ไหน — จับคู่ role กับ task type ด้วยภาษาธรรมชาติ พอ profile ไม่มี description ก็ยังส่ง name พร้อม fallback string ไปให้ LLM อยู่ดี แทนที่จะ omit

Normalize — ปิดช่องโหว่ hallucination

LLM อาจ hallucinate ชื่อ profile ที่ไม่มีจริง `_normalize_assignee_choice` จัดการ edge case นี้:

```
def _normalize_assignee_choice(
    assignee: object,
    *,
    default_assignee: str,
    valid_names: set[str],
) → str:
    """Return a valid assignee, falling back to ``default_assignee``.

    Fan-out children and the single-task fallback should share the same
    routing guarantee: promoted work must not be left unassigned.
    """
    if not isinstance(assignee, str) or not assignee.strip():
        return default_assignee
    chosen = assignee.strip()
    if chosen not in valid_names:
        return default_assignee
    return chosen
```

hermes_cli/kanban_decompose.py:252-268 @ 1899c8f50

พอ assignee ที่ LLM เลือกไม่อยู่ใน `valid_names` ก็ fall back หา `default_assignee` เสมอ — routing guarantee เป็น invariant ที่ enforce ก่อน task เข้า database

พอ LLM เลือก assignee ให้ child task แต่ละตัวแล้ว `decompose_triage_task` ใน `kanban_db.py` ก็สร้าง child tasks ทั้งหมดใน `todo` อย่าง atomic สร้าง DAG dependency tree โดยให้ root เป็น parent ของทุก child — พอ child ทุกตัว reach `done` ก็ promote root จาก `triage` → `todo` และ orchestrator profile wake up มาตัดสินใจว่า graph เสร็จจริงหรือยัง

I 6.2 dispatch_once — Atomic Claim ทุก 60 วินาที

พอ children อยู่ใน `todo` แล้ว dispatcher รับช่วงต่อ — `dispatch_once` ใน `kanban_db.py` ถูกเรียกในแต่ละ tick:

```
def dispatch_once(
    conn: sqlite3.Connection,
```

```

*,
spawn_fn=None,
ttl_seconds: Optional[int] = None,
dry_run: bool = False,
max_spawn: Optional[int] = None,
max_in_progress: Optional[int] = None,
failure_limit: int = DEFAULT_SPAWN_FAILURE_LIMIT,
stale_timeout_seconds: int = 0,
board: Optional[str] = None,
default_assignee: Optional[str] = None,
max_in_progress_per_profile: Optional[int] = None,
) → DispatchResult:
    """Run one dispatcher tick.

    Steps:
    1. Reclaim stale running tasks (TTL expired).
    2. Reclaim stale running tasks (no recent heartbeat).
    3. Reclaim crashed running tasks (host-local PID no longer alive).
    3. Promote todo → ready where all parents are done.
    4. For each ready task with an assignee, atomically claim and call
       ``spawn_fn(task, workspace_path, board) → Optional[int]``. The
       return value (if any) is recorded as ``worker_pid`` so subsequent
       ticks can detect crashes before the TTL expires.

```

hermes_cli/kanban_db.py:6025-6315 @ 1899c8f50

tick หนึ่งทำ 4 ขั้นตอนตามลำดับ: reclaim → promote → claim → spawn ขั้น reclaim สำคัญที่สุด
 — ก่อน dispatch งานใหม่ ระบบตรวจว่างานที่รันอยู่ยังมีชีวิตไหม ผ่านสามเงื่อนไข: TTL หมด, ไม่มี heartbeat, หรือ PID ไม่อยู่ใน process table อีกแล้ว

Atomic Claim — ไม่มีงานถูก dispatch สองครั้ง

claim_task ทำ atomic transition ready → running และสร้าง task_runs row:

```

def claim_task(
    conn: sqlite3.Connection,
    task_id: str,
    *,
    ttl_seconds: Optional[int] = None,

```

```

    claimer: Optional[str] = None,
) → Optional[Task]:
    """Atomically transition ``ready → running``.

    Returns the claimed ``Task`` on success, ``None`` if the task was
    already claimed (or is not in ``ready`` status).
    """
    ...

    run_cur = conn.execute(
        """
        INSERT INTO task_runs (
            task_id, profile, step_key, status,
            claim_lock, claim_expires, max_runtime_seconds,
            started_at
        ) VALUES (?, ?, ?, 'running', ?, ?, ?, ?)
        """,
        (
            task_id,
            trow["assignee"] if trow else None,
            trow["current_step_key"] if trow else None,
            lock,
            expires,
            trow["max_runtime_seconds"] if trow else None,
            now,
        ),
    )

```

hermes_cli/kanban_db.py:2972-3084 @ 1899c8f50

`profile` column ที่ insert คือ `trow["assignee"]` — ค่าเดียวกับที่ LLM เลือกตั้งแต่ตอน decompose นี่คือการตัดสินใจที่ routing decision ถูก seal ลงในประวัติ ทุก `task_runs` row บอกได้ว่า “claim โดย profile ไหน เวลาไหน expire เมื่อไหร่”

Concurrency Caps — ป้องกัน Hermes ล้น

`dispatch_once` มี 3 ระดับ cap:

```
spawned = 0
```

```
# Per-profile concurrency cap (#21582): when set, track how many
```

```

# workers each assignee already has in flight, and refuse to spawn
# when this would push that assignee past the cap. Prevents
# fan-out workloads from melting a single profile's local model /
# API quota / browser pool while leaving other profiles idle.
...
_per_profile_cap = max_in_progress_per_profile if (
    isinstance(max_in_progress_per_profile, int)
    and max_in_progress_per_profile > 0
) else None
...
# Respawn guard: refuse to re-spawn when useful work is already
# in-flight/recent, or when the last failure is a deterministic
# blocker (quota / auth). The guard defers the spawn this tick so
# the task gets a chance to clear (rate limits often reset in
# seconds-to-minutes); the existing consecutive_failures counter
# still trips the auto-block circuit breaker after failure_limit
# consecutive failures, so a persistent auth error eventually
# blocks via the normal path rather than on first occurrence.
guard_reason = check_respawn_guard(conn, row["id"])
if guard_reason is not None:
    result.respawn_guarded.append((row["id"], guard_reason))

```

hermes_cli/kanban_db.py:6100-6270 @ 1899c8f50

`max_spawn` คือ global live cap ของ tick นี้, `max_in_progress` คือ queue cap รวม, `max_in_progress_per_profile` คือไม่ให้ profile เดียว occupy ทุก slot — พอ fan-out task ทั้งหมด assign ให้ `researcher` คนเดียว ก็จะไม่กิน API quota ทั้งหมดในคราวเดียว ปล่อยให้ profile อื่นยังมีโอกาสรับงาน

respawn guard เป็น layer พิเศษ: พอ failure ล่าสุดเป็น quota หรือ auth error ก็ defer การ spawn แทน retry ทันที เพราะ rate limit มักรีเซ็ตเองในไม่ช้าก็ แต่ถ้า failure สะสมเกิน `failure_limit` ก็ auto-block ผ่าน circuit breaker — ป้องกัน infinite retry loop

I 6.3 Cross-Profile Spawn — hermes -p กับ Workspace Isolation

Workspace — Isolation ที่ถูก Design มา

`resolve_workspace` จัดการ 3 รูปแบบ:

```
def resolve_workspace(task: Task, *, board: Optional[str] = None) → Path:
    """Resolve (and create if needed) the workspace for a task.

    - ``scratch``: a fresh dir under ``<board-root>/workspaces/<id>/``,
      where ``<board-root>`` is the active board's root. The path is the
      same for the dispatcher and every profile worker, so handoff is
      path-stable.

    - ``dir:<path>``: the path stored in ``workspace_path``. Created
      if missing. MUST be absolute — relative paths are rejected to
      prevent confused-deputy traversal where ``../tmp/attacker``
      resolves against the dispatcher's CWD instead of a meaningful
      root. Users who want a kanban-root-relative workspace should
      compute the absolute path themselves.

    - ``worktree``: a git worktree at ``workspace_path``. Not created
      automatically in v1 -- the kanban-worker skill documents
      ``git worktree add`` as a worker-side step. Returns the intended path.
```

hermes_cli/kanban_db.py:4687-4749 @ 1899c8f50

`scratch` คือ default ที่ปลอดภัยที่สุด: filesystem path ถูกสร้างให้แต่ละ task_id โดยเฉพาะ และ path เดียวกันใช้ได้ทั้ง dispatcher กับ worker — handoff artifact เช่น downloaded file หรือ intermediate result อยู่ใน location เดียวกันโดยไม่ต้องส่งผ่าน message

`dir` ต้องเป็น absolute path เท่านั้น เพราะ relative path resolve ต่างกันใน dispatcher กับ worker (ซึ่งอาจรันใน cwd ต่างกัน) — ระบบ reject ก่อน spawn ทันที

`worktree` ใน v1 ไม่ถูกสร้าง auto — worker ต้องรัน `git worktree add` เอง เป็นการออกแบบที่ท้าทาย: ระบบไม่ assume ว่า task ทุกชั้นที่ใช้ worktree จะ checkout branch เดียวกัน

`_default_spawn` — Fire-and-Forget แบบ Traceable

พอ workspace resolve แล้ว dispatcher เรียก `_default_spawn`:

```
def _default_spawn(
    task: Task,
    workspace: str,
    *,
    board: Optional[str] = None,
) → Optional[int]:
    """Fire-and-forget ``hermes -p <profile> chat -q ...`` subprocess.
```

```

Returns the spawned child's PID so the dispatcher can detect crashes
before the claim TTL expires. The child's completion is still observed
via the ``complete`` / ``block`` transitions the worker writes itself;
the PID check is a safety net for crashes, OOM kills, and Ctrl+C.

...
"""

...
env["HERMES_KANBAN_TASK"] = task.id
env["HERMES_KANBAN_WORKSPACE"] = workspace

...
env["HERMES_KANBAN_DB"] = str(kanban_db_path(board=board))
env["HERMES_KANBAN_WORKSPACES_ROOT"] = str(workspaces_root(board=board))
resolved_board = _normalize_board_slug(board) or get_current_board()
env["HERMES_KANBAN_BOARD"] = resolved_board
env["HERMES_PROFILE"] = profile_arg
cmd = [
    *_resolve_hermes_argv(),
    "-p", profile_arg,
    "--accept-hooks",
]
...
proc = subprocess.Popen(
    cmd,
    cwd=workspace if os.path.isdir(workspace) else None,
    stdin=subprocess.DEVNULL,
    stdout=log_f,
    stderr=subprocess.STDOUT,
    env=env,
    start_new_session=True,
    creationflags=subprocess.CREATE_NO_WINDOW if _IS_WINDOWS else 0,
)
...
return proc.pid

```

hermes_cli/kanban_db.py:6678-6849 @ 1899c8f50

`-p profile_arg` กับ `env["HERMES_PROFILE"] = profile_arg` คือสิ่งที่ทำให้ “cross-engine” เกิดขึ้น
 จริง — `hermes -p researcher` บูท Hermes ด้วย configuration ของ `researcher` profile ซึ่งอาจมี

model ต่างกัน tool set ต่างกัน system prompt ต่างกัน ทั้งหมด controlled ด้วยชื่อ profile ตัวเดียวที่ LLM เลือกตั้งแต่ตอน decompose

env vars `HERMES_KANBAN_*` pin context ให้ worker: task id ที่ต้องรับผิดชอบ, workspace path ที่ต้องทำงาน, database path, board name — worker ใช้ env vars เหล่านี้รู้ว่าตัวเองควรรายงานผลผ่าน `complete_task` หรือ `block_task` ซึ่ง update `task_runs` row ที่ `claim_task` สร้างไว้ `start_new_session=True` ทำให้ worker detach จาก session ของ dispatcher — ถ้า dispatcher ตาย worker รันต่อได้ และถ้า worker crash ก็ไม่ถึง dispatcher ลงด้วย dispatcher คืบ `proc.pid` กลับมาเพื่อให้ tick ถัดไปตรวจว่า PID ยังมีชีวิตอยู่ไหม — พอ OOM kill พัด worker ออกโดยไม่ทัน write `complete_task` ก็ reclaim งานนั้นใน tick ถัดไปผ่าน PID check เลย

Complete — วงปิด

พอ worker ทำงานเสร็จ ก็เรียก `complete_task` ซึ่งทำสามสิ่ง: flip status `running` → `done`, clear `consecutive_failures`, และเรียก `recompute_ready` เพื่อให้ children ที่รอ parent ได้ promote เป็น `ready`:

```
def complete_task(
    conn: sqlite3.Connection,
    task_id: str,
    *,
    result: Optional[str] = None,
    summary: Optional[str] = None,
    metadata: Optional[dict] = None,
    ...
) → bool:
    """Transition ``running|ready → done`` and record ``result``.
    ...
    # Successful completion — wipe the consecutive-failures counter.
    # Failure history stays on the event log for audit; the counter
    # just tracks "is there a current pathology the breaker should
    # care about", and a success resets that question.
    _clear_failure_counter(conn, task_id)
    # Recompute ready status for dependents (separate txn so children see done).
    recompute_ready(conn)
```


hermes_cli/kanban_db.py:3562-3741 @ 1899c8f50

summary กับ metadata คือช่องทาง handoff สำหรับงาน multi-step: worker ตัวแรกเขียน

summary ของสิ่งที่ทำ, worker ตัวถัดไปอ่านผ่าน task_runs row ของ parent ก่อน start งานตัวเอง

recompute_ready มี guard สำคัญ: ถ้า blocked task มี consecutive_failures ≥ failure_limit ก็ skip การ promote ไม่ว่า parent จะ done แล้วหรือเปล่า — ป้องกัน cycle

block → auto-recover → spawn → fail → block → ... ไม่ให้วนซ้ำ

I Failure → Fix: workspace_kind Default อยู่ที่ไหน

ระหว่างการ research บกนี้ มีการ claim ว่า Task dataclass มี workspace_kind field ที่ default เป็น 'scratch' ใน Python level การ verify ไปดูในไฟล์จริงพบว่า dataclass field เป็นแค่ type annotation ไม่มี default:

ใน Python dataclass (line 746):

```
workspace_kind: str
```

ไม่มี = 'scratch' ในบรรทัดนั้น default ที่แท้จริงอยู่ใน SQL:

```
CREATE TABLE IF NOT EXISTS task_runs (  
  id                INTEGER PRIMARY KEY AUTOINCREMENT,  
  task_id           TEXT NOT NULL,  
  profile           TEXT,  
  step_key          TEXT,  
  status            TEXT NOT NULL,  
  -- status: running | done | blocked | crashed | timed_out | failed | released  
  claim_lock        TEXT,  
  claim_expires     INTEGER,  
  worker_pid        INTEGER,  
  max_runtime_seconds INTEGER,  
  last_heartbeat_at INTEGER,  
  started_at        INTEGER NOT NULL,  
  ended_at          INTEGER,  
  outcome           TEXT,  
  -- outcome: completed | blocked | crashed | timed_out | spawn_failed |  
  --             gave_up | reclaimed | (null while still running)
```

```
summary      TEXT,  
metadata      TEXT,  
error         TEXT  
);
```

hermes_cli/kanban_db.py:1073-1093 @ 1899c8f50

(และ `workspace_kind TEXT NOT NULL DEFAULT 'scratch'` อยู่ใน `CREATE TABLE tasks` statement ที่ line 985 ไม่ใช่ `dataclass`)

พอ code ไหนสร้าง `Task` object โดยไม่ส่ง `workspace_kind` เข้ามา ก็ raise `TypeError` ทันทันที — ไม่ใช้ default เป็น `'scratch'` อย่างเจี๊ยบๆ default อยู่ที่ database layer: พอ row ถูก insert โดยไม่ระบุ `workspace_kind` SQLite ก็ใส่ `'scratch'` ให้ แต่ Python code ที่ทำงานกับ `Task` object ต้องส่งค่ามาเองเสมอ

อย่า assume default จาก `dataclass` — ไปดู SQL schema ก่อนเสมอ

บทที่ 7: The Gateway & the 29 Adapters

Wire ทุกเส้น — Discord slash command, LINE webhook, Telegram photo burst, Slack mention, WhatsApp message, Matrix event, IoT HTTP call จาก ESP32 firmware — บรรจบที่

function เดียว: `handle_message()` ใน `gateway/platforms/base.py`

ไม่มี parser แยกต่างหากสำหรับแต่ละ platform ที่ชั้น pipeline หลัก ไม่มี if-else ยาวตรรกะว่า “ถ้ามาจาก Discord ทำอย่างนี้, ถ้ามาจาก Telegram ทำอย่างนั้น” มีแค่ funnel เดียว รับทุกอย่างเข้ามา ส่งออกไปเป็นคำตอบ

I 7.1 นับใหม่: 29 ไม่ใช่ 32

ตัวเลขที่ถูกคือ **29** — มาจากการนับจริงในโค้ดที่ commit `1899c8f50` ไม่ใช่จากความจำ แบ่งออกเป็นสองที่อยู่:

`gateway/platforms/` — **19 adapters (built-in):**

Adapter	Class
API Server	APIServerAdapter
BlueBubbles	BlueBubblesAdapter
DingTalk	DingTalkAdapter
Email	EmailAdapter
Feishu	FeishuAdapter
Matrix	MatrixAdapter
MS Graph Webhook	MSGraphWebhookAdapter
Signal	SignalAdapter
Slack	SlackAdapter
SMS	SmsAdapter
Telegram	TelegramAdapter
Webhook	WebhookAdapter
WeCom	WeComAdapter
WeCom Callback	WeComCallbackAdapter
Weixin	WeixinAdapter
WhatsApp	WhatsAppAdapter
WhatsApp Cloud	WhatsAppCloudAdapter
Yuanbao	YuanbaoAdapter
QQ	QQAdapter

plugins/platforms/ — 10 adapters (plugin layer):

Adapter	Class
Discord	DiscordAdapter
Google Chat	GoogleChatAdapter
Home Assistant	HomeAssistantAdapter
IRC	IRCAdapter
LINE	LineAdapter
Mattermost	MattermostAdapter
Ntfy	NtfyAdapter
Photon	PhotonAdapter
Simplex	SimplexAdapter
Teams	TeamsAdapter

$$19 + 10 = 29$$

ทุกตัวในสองกลุ่มนี้ inherit มาจาก base class เดียวกัน:

```
class BasePlatformAdapter(ABC):
```

gateway/platforms/base.py:1793 @ 1899c8f50

ABC คือ abstract base class — ไม่มีใครสร้าง `BasePlatformAdapter()` ตรงๆ ได้ พอ adapter ไม่ implement method ที่กำหนดไว้ให้ครบ ก็ compile ไม่ผ่านเลย — guarantee ว่าไม่มี adapter ไหน “ลืม” ใส่ `handle_message()` ได้

I 7.2 Three Layers ของ Entry Point

พอ message เข้ามา ก็ผ่าน 3 layer ตามลำดับ — ข้ามไม่ได้ ลัดวงจรไม่ได้

Layer 1: `BasePlatformAdapter.handle_message()` — base.py:3871

entry point สาธารณะที่ทุก adapter ใน 29 ตัวเรียกหรือ inherit มาใช้ — docstring บอกชัดว่า “returns quickly”:

```
async def handle_message(self, event: MessageEvent) → None:
    """
    Process an incoming message.

    This method returns quickly by spawning background tasks.
    This allows new messages to be processed even while an agent
    is running, enabling interruption support.
    """
```

gateway/platforms/base.py:3871-3878 @ 1899c8f50

“returns quickly” ไม่ใช่แค่ performance goal — คือ architectural guarantee พอ

`handle_message()` block อยู่ที่นี่ ก็คือ bug กันที่

พอ event เข้ามา function นี้ทำ 3 อย่างก่อนส่งต่อ:

หนึ่ง — build session key

```
session_key = build_session_key(
    event.source,
    group_sessions_per_user=self.config.extra.get(
        "group_sessions_per_user", True
    ),
    thread_sessions_per_user=self.config.extra.get(
```

```

        "thread_sessions_per_user", False
    ),
)

```

gateway/platforms/base.py:3889-3896 @ 1899c8f50

`session_key` คือ identifier ที่บอกว่า “user คนนี้ ใน chat นี้ บน platform นี้ = session เดียวกัน” — default `group_sessions_per_user=True` ทำให้แต่ละคนใน group chat มี session ของตัวเอง

สอง — auth bypass check

```

if should_bypass_active_session(cmd):
    ...
    response = await self._message_handler(event)

```

gateway/platforms/base.py:3943 @ 1899c8f50

`should_bypass_active_session()` ตรวจสอบ command text — พอเป็น `/status` หรือ `/cancel` ก็ route ข้ามไปได้เลย แม้ agent กำลัง run อยู่

สาม — delegate ไปยัง runner

`self._message_handler` คือ callback ที่ถูก inject ตอน startup:

```

adapter.set_message_handler(self._handle_message)

```

`self._handle_message` คือ method ใน gateway runner — layer ถัดไป

Layer 2: GatewayRunner._handle_message() — run.py:6362

core pipeline จริงๆ — docstring ของมันบอกขั้นตอนทั้งหมด:

```

async def _handle_message(
    self, event: MessageEvent
) → Optional[str]:
    """
    Handle an incoming message from any platform.

    This is the core message processing pipeline:
    1. Check user authorization
    2. Check for commands (/new, /reset, etc.)
    3. Check for running agent and interrupt if needed
    4. Get or create session
    """

```

```
5. Build context for agent
6. Run agent conversation
7. Return response
"""
```

gateway/run.py:6362-6374 @ 1899c8f50

7 ขั้นตอนตามลำดับ ไม่มีขั้นตอนไหนที่ข้ามได้โดยไม่ตั้งใจ

ขั้นที่ 3 คือจุดที่ interrupt ทำงาน — พอ user ส่ง message ใหม่ขณะ agent กำลังคิดอยู่ ก็ตรวจว่า

`_busy_input_mode = "interrupt"` (ค่า default) แล้ว call `agent.request_interrupt()` ทั้งนี้ —

agent loop poll flag นี้แล้วหยุดตัวเองก่อน return partial result

ก่อนที่ message จะถึงขั้น “run agent” ต้องผ่าน concurrent session gate ก่อน:

```
self._running_agents[_quick_key] = _AGENT_PENDING_SENTINEL
self._running_agents_ts[_quick_key] = time.time()
_run_generation = self._begin_session_run_generation(_quick_key)

try:
    _agent_result = await self._handle_message_with_agent(
        event, source, _quick_key, _run_generation
    )
```

gateway/run.py:7546-7551 @ 1899c8f50

`_AGENT_PENDING_SENTINEL` ถูกใส่ใน `_running_agents` dict ก่อนที่ agent จะ start — ป้องกัน race condition ที่ message สองอันมาพร้อมกันแล้วทั้งคู่คิดว่า session ว่างอยู่

`_run_generation` คือ counter ที่ increment ทุกครั้งที่ session เริ่ม run ใหม่ — พอ user interrupt แล้วส่ง message ใหม่มา generation counter เปลี่ยน ก็ discard result จาก agent เก่าโดยอัตโนมัติ

Layer 3: `_handle_message_with_agent()` — run.py:7881

agent loop จริงๆ ที่ทำงานใต้ sentinel guard:

```
async def _handle_message_with_agent(
    self,
    event,
    source,
```

```

    _quick_key: str,
    run_generation: int,
):
    """Inner handler that runs under the _running_agents sentinel guard."""
    _msg_start_time = time.time()
    _platform_name = (
        source.platform.value
        if hasattr(source.platform, "value")
        else str(source.platform)
    )
    _msg_preview = (event.text or "")[:80].replace("\n", " ")
    logger.info(
        "inbound message: platform=%s user=%s chat=%s msg=%r",
        _platform_name,
        source.user_name or source.user_id or "unknown",
        source.chat_id or "unknown",
        _msg_preview,
    )

```

gateway/run.py:7881-7890 @ 1899c8f50

log line แรกบอกทุกอย่างที่ต้องรู้เพื่อ debug: platform ชื่ออะไร, user คือใคร, chat ID คืออะไร, message ขึ้นต้นว่าอะไร

จาก `_handle_message_with_agent()` ลงไปอีก: - message text อาจถูก enrich ด้วย vision analysis (ถ้า user ส่งรูป) - หรือ transcription (ถ้า user ส่ง voice message) - session history ถูก load มา - context prompt ถูก build - แล้วค่อย call `agent.run_conversation()` ทั้ง 3 layer working together ทำให้ gateway รองรับได้พร้อมกัน: - multiple users บน platform เดียวกัน (แต่ละคนมี `_quick_key` ต่างกัน) - multiple platforms พร้อมกัน (แต่ละ adapter run event loop ของตัวเอง) - interrupt + resume ใน session เดียว (ผ่าน generation counter)

7.3 The 3-Second Problem: Discord Adapter

Discord's API กำหนดว่า พอ bot ไม่ respond ต่อ slash command ภายใน 3 วินาที ก็แสดง

“application did not respond” ทันที — แม้ bot กำลังทำงานอยู่

agent loop ของ Hermes ใช้เวลานานกว่า 3 วินาทีเสมอ บางครั้ง 30 วินาที บางครั้ง 65 วินาที — พอ

Discord adapter รอให้ `handle_message()` เสร็จก่อนค่อย respond ก็ทุก slash command fail เลย

วิธีแก้คือ **defer pattern** ที่ `_run_simple_slash()` ใน

`plugins/platforms/discord/adapters.py:3316`:

```
# Auth gate — must run before defer() so an ephemeral rejection
# can be delivered on the still-unresponded interaction.
if not await self._check_slash_authorization(interaction, command_text):
    return

await interaction.response.defer(ephemeral=True)
event = self._build_slash_event(interaction, command_text)
await self.handle_message(event)
```

`plugins/platforms/discord/adapters.py:3316-3323` @ 1899c8f50

Auth gate ต้องมาก่อน `defer()` — `interaction.response.defer()` คือการ “ตอบรับ” interaction แล้ว พอ `defer()` ไปแล้วก็ส่ง ephemeral error กลับไม่ได้อีก เพราะ interaction ถูก respond ไปแล้ว flow ที่ถูกต้อง: 1. ตรวจสอบ authorization ก่อน — ถ้าไม่ผ่าน ส่ง ephemeral error แล้วออก 2. ถ้าผ่านค่อย defer (แสดง “thinking...” ให้ user เห็น) 3. dispatch ไปยัง `handle_message()` ซึ่ง return fast (spawns background task)

พอ `defer()` แล้ว ก็มีเวลาถึง 15 นาทีในการ call `interaction.edit_original_response()` เพื่อส่งคำตอบจริงๆ กลับไป — ยาวพอสำหรับ agent loop ที่ซับซ้อนที่สุด

Slack adapter ที่ `gateway/platforms/slack.py:2744` เรียก `handle_message()` ตรงๆ โดยไม่มี explicit defer:

```
await self.handle_message(msg_event)
```

`gateway/platforms/slack.py:2744` @ 1899c8f50

Slack รับ HTTP POST แล้ว respond กลับโดยตรง — ไม่ใช่ WebSocket interaction เหมือน Discord เลยไม่ต้องการ defer pattern เดียวกัน ความแตกต่างนี้สะท้อน tradeoff ของ architecture: built-in adapters ใน `gateway/platforms/` simple กว่า plugin adapters ที่แบก platform-specific quirks ไว้เอง

I สิ่งที่ Verified จริงๆ ใน Session นี้

- `BasePlatformAdapter.handle_message()` ที่ `base.py:3871` — **confirmed**: returns by spawning background tasks
- session keying ผ่าน `build_session_key()` — **confirmed**: ที่ `base.py:3889-3896`
- `_handle_message()` core pipeline ที่ `run.py:6362` — **confirmed**: 7 steps ในลำดับที่แน่นอน
- `_running_agents` sentinel ที่ `run.py:7546` — **confirmed**: ป้องกัน concurrent execution ใน session เดียว
- `_handle_message_with_agent()` ที่ `run.py:7881` — **confirmed**: agent loop ที่ run ใต้ sentinel guard
- Discord defer pattern ที่ `plugins/platforms/discord/adapter.py:3316` — **confirmed**: auth gate ก่อน defer เสมอ
- จำนวน adapters — **confirmed by count**: 19 ใน gateway + 10 ใน plugins = 29

open questions สำหรับ session ถัดไป: - Slack slash command ACK mechanism (ต่างจาก Discord ยังไง?) - `WeComCallbackAdapter` นับเป็น independent platform หรือ companion adapter? - `hermes_cli/commands.py` เชื่อมกับ agent loop ยังไง?

I Failure → Fix: 32 → 29

ใน session ที่ผ่านมามีคำตอบว่า “32 adapters” — ตัวเลขนั้นมาจากความจำ ไม่ใช่จากการนับจริง พอนับจริงโดยใช้ code archaeology — ดู directory listing ของ `gateway/platforms/` และ `plugins/platforms/` แล้วนับ class ที่ inherit `BasePlatformAdapter` — ก็ได้ $19 + 10 = 29$

```
class BasePlatformAdapter(ABC):
```

`gateway/platforms/base.py:1793 @ 1899c8f50`

```
class DiscordAdapter(BasePlatformAdapter):
```

`plugins/platforms/discord/adapter.py:598 @ 1899c8f50`

ตัวเลข “จากความจำ” เกี่ยวกับโค้ดผิดเสมอ — นับจากโค้ดจริงเท่านั้น